Part 1 – Question Paper Creation

Section A: 10-Mark Questions

1. Explain Relational Database Management Systems (RDBMS) and SQL, contrast SQL with programming languages like Python and R for analytics, and detail the architectural features and capabilities of SQLite.
2. Discuss Data Definition Language (DDL) and Data Manipulation Language (DML) in detail, providing SQL syntax and practical statement examples for table creation, schema modification, data insertion, updating, and row removal.
3. Explain SQL Joins in detail, including Inner Join, Outer Joins (Left, Right, Full Outer), Self Join, and Cross Join, detailing their syntax, logical execution, and usage scenarios.
4. Discuss SQL Aggregate Functions, Data Grouping using GROUP BY, Group Filtering using HAVING, and Conditional Logic using CASE WHEN statements with syntax and practical analytical examples.

Section B: 5-Mark Questions

5. Explain the ACID properties in database management systems and detail how Transaction Control Language (TCL) statements maintain transaction integrity.
6. Compare and contrast the WHERE clause and HAVING clause in SQL, providing a tabular summary and query examples.
7. Explain Database Integrity Constraints and contrast Primary Keys with Foreign Keys using a comparison table.
8. Describe Indexes in SQL, detailing B-Tree data structures, secondary indexes, and query performance optimization guidelines.
9. Explain Derived Data and Materialized Views, and compare Materialized Views with standard SQL Views using a comparison table.
10. Explain SQL search filtering operators (BETWEEN, IN, LIKE string pattern matching) and the order of precedence for Boolean logical operators (NOT, AND, OR).
11. Discuss SQL Subquery operators (EXISTS, ANY, ALL) and Common Table Expressions (CTEs) using the WITH clause.
12. Explain record modification and deletion operations (UPDATE and DELETE FROM), highlighting techniques to optimize performance.

Section C: 2-Mark Questions

13. Define Structured Query Language (SQL) and state its historical origin and standardized adoption.
14. What is SQLite, and what are its maximum storage limits for database size and row size?
15. Compare DROP TABLE and DELETE FROM statements in SQL.
16. Explain the purpose and operation of the DISTINCT keyword in a SELECT query.
17. Explain the structural difference between using single quotes and backticks in SQL query syntax.
18. Define Data Normalization and state its primary benefit in relational database design.
19. Describe the function of the ORDER BY clause and its default sorting order.
20. Define a CROSS JOIN and explain the mathematical output it generates.
21. Explain the role of SAVEPOINT and ROLLBACK TO SAVEPOINT in transaction control.
22. Contrast INSERT INTO ... SELECT with SELECT INTO when copying data between tables.

Part 2 – Detailed Answer Bank

Section A: 10-Mark Answer Bank

Question 1: RDBMS Foundations, SQL vs. Python/R, and SQLite Architecture

Introduction: Relational Database Management Systems (RDBMS) organize structured data into logical entities, while SQL serves as the universal declarative language to interact with these systems for data analytics.

- **Definition of RDBMS:** A Relational Database Management System is a software framework that structures data around business entities and logical organizational information needs. It arranges data into structured tables composed of rows and columns, establishing logical relationships across distinct data entities. This structure allows organizations to efficiently store, organize, and retrieve complex datasets.
- **Introduction to SQL:** Structured Query Language (SQL, pronounced S-Q-L or sequel) is the standard computer language used to query, manipulate, and manage relational databases. Developed at IBM in the early 1970s and standardized by ANSI/ISO in 1986, SQL provides a universal framework to ask questions of a database and receive structured answers.
- **Role of SQL in Data Analytics:** Analytics involves discovering, interpreting, and communicating meaningful patterns in data to support effective organizational decision-making. SQL serves as a primary tool for data engineers, database administrators, and analysts to extract, clean, and aggregate raw database records for reporting.
- **SQL vs. Programming Languages (Syntax & Logic):** Unlike general programming languages like Python or R, which involve complex syntax, algorithmic structures, and steeper learning curves for beginners, SQL is simple and highly accessible. SQL is declarative, allowing analysts to declare *what* data to retrieve rather than writing custom procedural loops to fetch it.
- **Complementary Analytical Ecosystem:** SQL and procedural programming languages like Python or R complement each other rather than act as direct competitors. SQL excels at direct database access, relational data manipulation, and high-speed aggregation, whereas Python and R specialize in advanced statistics, custom procedural logic, and machine learning.
- **Definition of SQLite:** SQLite is an in-process, self-contained, serverless, and zero-configuration transactional database engine. It embeds directly into client applications as a lightweight library, eliminating the administrative overhead of deploying and configuring a separate background database server process.
- **Single-File Storage Architecture:** SQLite encapsulates an entire relational database—including tables, indexes, triggers, and schema definitions—within a single, standard cross-platform file on disk. This enduring file format ensures high portability, allowing users to transfer database instances across systems or back them up by copying a single file.
- **Scale & Storage Capacities of SQLite:** Despite its small footprint and serverless nature, SQLite supports massive data scale suitable for desktop and enterprise analytics. It handles maximum database file sizes of up to 140 terabytes and supports individual row sizes of up to 1 gigabyte.
- **Performance and Software Quality:** SQLite delivers fast execution speed, outperforming direct OS file access operations while maintaining a minimal memory footprint. Engineered with aviation-grade testing standards, public domain source code, and long-term support committed through the year 2050, it offers an ideal environment for learning and deploying analytics.

- **ACID Compliance in SQLite:** SQLite provides full ACID (Atomicity, Consistency, Isolation, and Durability) transactional guarantees, ensuring complete data integrity even after unexpected system crashes or power losses. This ensures that complex database updates complete entirely or roll back safely without corrupting storage.

Question 2 Data Definition Language (DDL) and Data Manipulation Language (DML)

Introduction: SQL commands are categorized into core functional sub-languages, primarily Data Definition Language (DDL) for defining database schema objects and Data Manipulation Language (DML) for managing database records.

- **Overview of Data Definition Language (DDL):** DDL statements define, modify, and drop database structures such as tables, schemas, and indexes. DDL operations directly alter the system catalog dictionary and automatically commit structural changes to persistent storage.
- **Creating Tables (CREATE TABLE):** The CREATE TABLE command defines new table schemas by specifying column identifiers, data types (INTEGER, VARCHAR, TEXT, NUMERIC, DATE), and integrity constraints.

```
CREATE TABLE MOSQUITO_TRAP_DATA (
   SAMPLEID INTEGER PRIMARY KEY AUTOINCREMENT,
   TRAP_DATE NUMERIC,
   GENUS TEXT,
   SPECIES TEXT,
   TOTAL INTEGER
);
```

This statement initializes a structured table schema with primary key constraints.

- **Modifying Schemas (ALTER TABLE):** The ALTER TABLE statement dynamically modifies existing table structures without destroying existing row records.
    - ALTER TABLE MOSQUITO_TRAP_DATA ADD COLUMN RURALNORTHWEST INTEGER;
    - ALTER TABLE MOSQUITO_TRAP_DATA DROP COLUMN RURALNORTHWEST;

This command allows schemas to evolve alongside changing business requirements.

- **Removing Table Schemas (DROP TABLE):** The DROP TABLE command completely purges an existing table schema along with all contained row data and associated indexes from the database catalog.
    - DROP TABLE MOSQUITO_TRAP_DATA;
    - Unlike row deletion, dropping a table permanently removes the table's structural metadata.
- **Overview of Data Manipulation Language (DML):** DML statements operate on individual data tuples within existing tables, enabling insertion, modification, deletion, and query retrieval of records. DML operations run within transactional contexts and can be rolled back if uncommitted.
- **Inserting Records (INSERT INTO):** The INSERT INTO command appends new rows of data into a designated table by mapping target column lists to explicit values.

  ```
  INSERT INTO MOSQUITO_TRAP_DATA (TRAP_DATE, GENUS, SPECIES, TOTAL)
  VALUES ('2014-07-01', 'Aedes', 'vexans', 290);
  ```

  This inserts new data records while enforcing defined column constraints.
- **Updating Existing Records (UPDATE):** The UPDATE statement modifies existing attribute values across table rows matching specific logical conditions.

  ```
  UPDATE MOSQUITO_TRAP_DATA
  ```

SET TOTAL = 300
WHERE SPECIES = 'vexans';
Updating selective records using precise WHERE clauses optimizes execution performance.

- **Deleting Rows (DELETE FROM):** The DELETE FROM statement removes specific data rows satisfying a logical predicate while maintaining the table schema.
    - DELETE FROM MOSQUITO_TRAP_DATA WHERE GENDER = 'Male';

  Omitting the WHERE clause clears all rows from the table while leaving the empty schema intact.
- **Querying Records (SELECT):** The SELECT command forms the foundation of data querying by retrieving specified columns and rows from database tables.

```
SELECT GENUS, SPECIES, TOTAL
FROM MOSQUITO_TRAP_DATA
WHERE TOTAL > 50
ORDER BY TOTAL DESC;
```

  This statement retrieves, filters, and orders records according to analytical criteria.
- **Copying Records Across Tables:** SQL allows bulk copying of records between tables using INSERT INTO ... SELECT for existing destination tables or SELECT INTO for creating new destination tables dynamically.

```
INSERT INTO TRAP_ARCHIVE
SELECT * FROM MOSQUITO_TRAP_DATA WHERE TOTAL > 100;
```

  This command streamlines data staging, archiving, and ETL batch transformations.

Question 3 SQL Joins (Inner, Left, Right, Full Outer, Self, Cross Join)

Introduction: A SQL JOIN brings together related records across multiple database tables based on shared key references, consolidating distributed data entities for relational analysis.

- **Concept and Purpose of Joins:** Joins link records across tables that share common attribute references, typically connecting foreign keys to primary keys. This relational operation frees databases from single-table constraints, enabling multi-entity reporting.
- **Inner Join (INNER JOIN) Mechanics:** An Inner Join evaluates two tables and returns only those row pairs where matching values exist across specified join key columns. Unmatched rows from either table are excluded from the output result set.

```
SELECT LED_STREETLIGHT.STREETLIGHT_ID, LED_STREETLIGHT.TYPE,
LOCATION.LOCATION
FROM LED_STREETLIGHT
INNER JOIN LOCATION ON LED_STREETLIGHT.STREETLIGHT_ID =
LOCATION.STREETLIGHT_ID;
```

  This returns streetlight records only when matching location entries exist.
- **Left Outer Join (LEFT JOIN) Mechanics:** A Left Outer Join preserves all records from the left table, pairing them with matching rows from the right table. If no matching row exists in the right table, NULL values are populated across right table attributes.

```
SELECT A.id, B.status
FROM customers A
LEFT JOIN orders B ON A.id = B.customer_id;
```

  This preserves all customer records regardless of whether they have matching orders.
- **Right Outer Join (RIGHT JOIN) Mechanics:** A Right Outer Join preserves every record from the right table while bringing in matching rows from the left table. Unmatched attributes from the left table are populated with NULL.

- **Full Outer Join (FULL OUTER JOIN) Mechanics:** A Full Outer Join preserves all unmatched records from both left and right tables simultaneously. Rows with matching keys are merged, while unmatched rows from either side populate missing partner attributes with NULL.
- **Self Join Concepts:** A Self Join occurs when a table is joined directly to itself to compare rows within the same dataset. This operation requires table aliasing to differentiate the two functional instances of the table during execution.
- **Self Join Execution Example:** Self joins calculate temporal differences or sequential relationships between rows within a single table.

```
SELECT previous.ex_date AS start, executions.ex_date AS end,
    ULIANDAY(executions.ex_date) - JULIANDAY(previous.ex_date) AS day_difference
FROM executions
JOIN executions previous ON executions.ex_number = previous.ex_number + 1;
```

  Here, table aliasing executions previous enables row-by-row comparisons.
- **Multiple Join Conditions:** Queries can specify multiple logical predicates within an ON clause using AND operators to join tables across multiple attributes simultaneously.

```
SELECT a.id, MIN(b.timestamp) - a.timestamp AS response_time
FROM emails a
JOIN emails b ON b.subject = a.subject
        AND a.to = b.from
        AND a.from = b.to
        AND a.timestamp < b.timestamp
WHERE a.to = 'zach@g.com'
GROUP BY a.id;
```

  This multi-predicate join matches complex communication thread flows.
- **Cross Join (CROSS JOIN) / Cartesian Product:** A Cross Join combines every individual row of the first table with every individual row of the second table, producing a Cartesian product. Joining a table of M rows with a table of N rows yields M x N output rows.
- **Multiple Matches and Table Aliasing:** When join keys match multiple rows across tables, the database engine generates output row combinations for every valid match pair. Explicit table aliasing (table.column) is necessary to prevent column ambiguity errors in result sets.

Question 4 Aggregate Functions, Data Grouping (GROUP BY), HAVING, and CASE WHEN

Introduction: SQL aggregate functions summarize multiple row values into single summary metrics, while GROUP BY, HAVING, and CASE WHEN statements enable categorical grouping, post-aggregation filtering, and conditional logic.

- **Concept of Aggregation:** Aggregation combines multiple data rows into a single numerical summary value. Aggregate functions are essential in data analytics for summarizing transactional rows into key performance indicators.
- **Standard SQL Aggregate Functions:** SQL provides standard aggregate functions to compute statistics over dataset columns:
  - COUNT(): Counts total rows or non-null column entries.
  - SUM(): Calculates the total arithmetic sum of numeric values.
  - AVG(): Computes the arithmetic mean of numeric values.
  - MIN() / MAX(): Finds minimum or maximum column values.
- **NULL Value Handling in Aggregates:** Aggregate functions ignore NULL values when computing column metrics, except for COUNT(*) which counts all table rows. Understanding NULL handling prevents skewed statistical calculations.

- **Categorical Grouping (GROUP BY Block):** The GROUP BY clause splits datasets into distinct groups and applies aggregate functions independently per group, returning one summary row per category.

```
SELECT GENUS, SUM(TOTAL) AS total_mosquitos
FROM MOSQUITO_TRAP_DATA
GROUP BY GENUS;
```

  This query computes total catches categorized by mosquito genus.
- **Syntax Order and Execution Precedence:** Syntactically, the GROUP BY clause follows the WHERE clause. Logically, row-level filtering via WHERE executes *before* grouping occurs, restricting the inputs processed by GROUP BY.
- **Non-Aggregated Column Restrictions:** Non-aggregated columns listed in a SELECT clause must be explicitly declared in the GROUP BY block. Mixing non-grouped, non-aggregated columns causes execution errors or ambiguous outputs.
- **Post-Aggregation Filtering (HAVING Block):** The HAVING clause filters result sets based on aggregate values computed *after* GROUP BY processing completes. Since WHERE cannot evaluate aggregate functions, HAVING supplies necessary post-group filtering.

```
SELECT GENUS, SUM(TOTAL) AS grand_total
FROM MOSQUITO_TRAP_DATA
GROUP BY GENUS
HAVING SUM(TOTAL) > 100;
```

  This restricts output to genera with total catches exceeding 100.
- **Conditional Logic (CASE WHEN Statement):** The CASE WHEN block functions as an inline if-else conditional evaluation structure inside SQL queries, returning specific values when specified boolean conditions evaluate true.

```
CASE
   WHEN condition1 THEN result1
   WHEN condition2 THEN result2
   ELSE default_result
END
```

  Omitting the ELSE clause returns NULL when all tested conditions evaluate false.
- **Conditional Aggregation:** CASE WHEN statements can be nested inside aggregate functions to compute multiple conditional metrics within a single pass query.

```
SELECT
   SUM(CASE WHEN county = 'Harris' THEN 1 ELSE 0 END) AS harris_count,
   SUM(CASE WHEN county = 'Bexar' THEN 1 ELSE 0 END) AS bexar_count
FROM executions;
```

  This tallies separate category totals into distinct result columns.
- **Cumulative Aggregations & Window Functions:** SQL supports cumulative metrics across ordered sets using self-joins or modern window functions like SUM() OVER ().

```
SELECT date, cash_flow,
     SUM(cash_flow) OVER (ORDER BY date ASC) AS cumulative_cf
FROM transactions
ORDER BY date ASC;
```

  Window functions compute running totals efficiently without collapsing individual rows.

Section B: 5-Mark Answer

Question 5 ACID Properties and Transaction Control Language (TCL)

Introduction: Relational database transactions preserve data integrity through ACID properties, which are managed using Transaction Control Language (TCL) statements.

- **Atomicity & Consistency Principles:** Atomicity guarantees that all statements within a transaction execute successfully or fail entirely, preventing partial updates. Consistency ensures that transactions transition the database between valid states, upholding all schema constraints.
- **Isolation & Durability Principles:** Isolation ensures concurrent transactions execute independently without exposing intermediate uncommitted states. Durability guarantees that committed transaction changes permanently persist in storage, even during system crashes.
- **Saving Changes (COMMIT):** The COMMIT statement permanently writes all data modifications made during the current transaction to disk storage. Once committed, changes become durable and visible across database sessions.
- **Reverting Changes (ROLLBACK):** The ROLLBACK statement cancels uncommitted data modifications performed during the active transaction. It restores table records to their pre-transaction state, protecting storage from operational errors.
- **Checkpoints & Modes (SAVEPOINT & SET TRANSACTION):** The SAVEPOINT command establishes intermediate markers within complex transactions, allowing partial rollbacks via ROLLBACK TO SAVEPOINT. SET TRANSACTION configures transaction properties, such as setting READ ONLY or READ WRITE access modes.

Question 6 Comparison of WHERE clause and HAVING clause

Introduction: Both WHERE and HAVING filter data in SQL queries, but they execute at different stages of query processing and operate on distinct levels of data granularity.

Comparison Table (WHERE vs. HAVING):

Feature	WHERE Clause	HAVING Clause
Execution Stage	Evaluated *before* data grouping and aggregation occur.	Evaluated *after* GROUP BY processing completes.
Filtering Target	Filters individual input rows from database tables.	Filters aggregated group summary results.
Aggregate Functions	Cannot evaluate aggregate functions (e.g., SUM, COUNT).	Explicitly designed to evaluate aggregate function outputs.
Syntax Placement	Placed directly before the GROUP BY clause.	Placed directly after the GROUP BY clause.
Usage Context	Used across standard single-table and multi-table queries.	Used alongside GROUP BY aggregation queries.

- **Row-Level Filtering (WHERE):** The WHERE clause screens raw table rows prior to grouping, evaluating predicates against individual row attributes. It reduces the volume of records passed downstream to aggregate functions.
- **Group-Level Filtering (HAVING):** The HAVING clause acts as a post-filter operating exclusively on group summary rows generated by a GROUP BY block. It screens grouped metrics calculated dynamically during query runtime.

- **Aggregate Restrictions in WHERE:** Including aggregate expressions like WHERE SUM(TOTAL) > 50 causes syntax errors because aggregate totals do not exist when individual rows are processed. WHERE references raw column attributes.
- **Combined Usage Query Example:**

```
SELECT GENUS, SUM(TOTAL) AS total_catch
FROM MOSQUITO_TRAP_DATA
WHERE TYPE = 'Banded legs'
GROUP BY GENUS
HAVING SUM(TOTAL) > 10;
```

Here, WHERE filters individual rows by leg type, while HAVING filters genus groups by aggregate catch size.

Question 7: Database Integrity Constraints and Key Comparison

Introduction: Integrity constraints enforce business rules and data consistency in relational databases, ensuring invalid data cannot be stored.

Primary Key vs. Foreign Key Comparison Table:

Constraint Feature	Primary Key (PRIMARY KEY)	Foreign Key (FOREIGN KEY)
Core Purpose	Uniquely identifies each individual row within a table.	Establishes relationships by referencing another table's key.
Uniqueness & NULLs	Enforces strict NOT NULL and UNIQUE constraints.	Allows duplicate entries and NULL values unless restricted.
Quantity Limits	Exactly one primary key is permitted per table schema.	Multiple foreign keys can be defined per table.
Reference Direction	Referenced by foreign keys in child tables.	References primary keys or unique keys in parent tables.

- **Primary Key & Uniqueness (PRIMARY KEY & UNIQUE):** A Primary Key combines NOT NULL and UNIQUE constraints to uniquely identify every row record. The UNIQUE constraint guarantees distinct column values while allowing a single NULL entry.
- **Foreign Keys & Referential Integrity (FOREIGN KEY):** A Foreign Key enforces referential integrity by requiring column values to match an existing primary key in a related parent table. This prevents orphaned child records.
- **Value Validation (NOT NULL & CHECK):** The NOT NULL constraint prevents blank or missing entries in mandatory fields. The CHECK constraint verifies that column values satisfy specific boolean conditions (e.g., TOTAL >= 0).
- **Defaults & Search Structures (DEFAULT & INDEX):** The DEFAULT constraint automatically inserts a preset value when no explicit value is supplied during insertion. The INDEX constraint creates internal lookup structures to speed up data retrieval.

Question 8 Database Indexes, B-Trees, and Performance Optimization

Introduction: Database indexes are specialized physical data structures that accelerate record retrieval and optimize query execution speed.

- **Indexing Fundamentals:** An index reorganizes references to unordered table data into a structured order using specific column values. It creates a search structure that allows the database engine to use fast search algorithms like binary search.
- **B-Tree Data Structures:** Indexes commonly utilize self-balancing B-Tree or B+Tree search data structures. B-Trees maintain sorted column values and support searching, sequential access, insertions, and deletions in logarithmic time (\(O(\log n)\)).

- **Secondary Indexes:** A secondary index is an additional data structure maintained alongside primary table storage. It enables fast searching on non-primary key columns that are frequently filtered or sorted.
- **Index Operations Syntax:** Indexes are created using CREATE INDEX (or CREATE UNIQUE INDEX) and removed using DROP INDEX.
    - CREATE INDEX idx_species ON MOSQUITO_TRAP_DATA (SPECIES);
    - DROP INDEX idx_species;

  Indexes remain invisible to end-users and are utilized automatically by query optimizers.
- **Performance Optimization Guidelines:** Maintaining indexes adds write overhead during INSERT, UPDATE, and DELETE operations. Optimization best practices include disabling indexes during bulk data loads, avoiding indexes on columns with low selectivity, using partial indexes, and applying Block Range Indexes (BRIN) for correlated data.

Question 9 Derived Data, Materialized Views, and Standard Views

Introduction: Derived datasets speed up read access by storing computed query results, with views and materialized views serving as key architectural abstractions.

Standard Views vs. Materialized Views Comparison Table:

Aspect	Standard View (VIEW)	Materialized View (MATERIALIZED VIEW)
Data Persistence	Virtual table; stores only the underlying SELECT query definition.	Physical table; computes and persists actual result data to disk.
Query Execution	Re-executes the underlying SQL query every time the view is queried.	Reads pre-computed results directly from disk storage.
Read Performance	Slower for complex aggregations and multi-table joins.	High-speed read performance for heavy aggregations.
Write Impact	No write penalty when updating base table records.	Adds write overhead as persisted view data must be updated.

- **Concept of Derived Data:** Derived data refers to secondary datasets generated from raw underlying data through repeatable computations. Examples include indexes, caches, and materialized views designed to optimize read access.
- **Standard SQL Views:** A standard view is a virtual table defined by an underlying SQL SELECT query. It encapsulates complex query logic and simplifies data access without consuming additional physical storage.
    - CREATE VIEW high_trap_counts AS
    - SELECT GENUS, SPECIES, TOTAL FROM MOSQUITO_TRAP_DATA WHERE TOTAL > 100;

  This creates a reusable virtual abstraction over the trap dataset.
- **Materialized Views Mechanics:** Materialization computes query results preemptively and writes the output directly to disk storage. This allows subsequent queries to bypass expensive runtime joins and aggregations.
- **Architectural Trade-offs:** Materialized views improve read performance for analytical reporting workloads, but they introduce write penalties because base table updates require refreshing persisted view data.

Question 10 Search Filtering Operators (BETWEEN, IN, LIKE) & Logical Precedence

Introduction: Advanced filtering operators refine SQL query predicates, while logical precedence rules dictate how multi-condition search clauses are evaluated.

- **Inclusive Range Filtering (BETWEEN):** The BETWEEN operator filters records matching an inclusive numerical, text, or date range.
    - SELECT product FROM inventory WHERE amount BETWEEN 5 AND 9;

  This query is logically equivalent to writing WHERE amount >= 5 AND amount <= 9.
- **List Value Matching (IN):** The IN operator filters rows whose column value matches any discrete entry within an explicit value list or subquery output.
    - SELECT product FROM inventory WHERE amount IN (1, 5, 10);

  IN simplifies query syntax by replacing multiple OR equality comparisons.
- **Pattern Matching (LIKE & Wildcards):** The LIKE operator performs pattern matching on text columns using wildcard characters. The % wildcard matches zero or more arbitrary characters, whereas _ matches exactly one character.
    - SELECT * FROM executions WHERE last_name LIKE '%Landry%';

  This retrieves rows where last_name contains "Landry" anywhere in the string.
- **Quotes vs. Backticks in Syntax:** Literal string values must be enclosed within single quotes (e.g., 'Banded legs'), with uppercase letters evaluating dictionary-wise smaller than lowercase letters. Backticks denote table or column identifiers that contain spaces or match SQL keywords.
- **Boolean Precedence (NOT > AND > OR):** When evaluating complex WHERE clauses containing multiple conditions, SQL enforces a default order of precedence: NOT is evaluated first, followed by AND, and finally OR. Parentheses () override default precedence.

Question 11 Subquery Operators (EXISTS, ANY, ALL) and Common Table Expressions (CTEs)

Introduction: Subqueries and Common Table Expressions (CTEs) enable modular, nested SQL query structures to solve multi-stage analytical tasks.

- **Existence Checks (EXISTS):** The EXISTS subquery operator tests whether a subquery returns any result rows, evaluating to TRUE as soon as a single matching record is identified.

  SELECT column_name FROM table_name
  WHERE EXISTS (SELECT column_name FROM table_name WHERE condition);

  It optimizes execution by stopping subquery scanning immediately upon finding the first match.
- **Quantified Comparison Operators (ANY & ALL):** The ANY operator returns TRUE if a comparison holds for at least one value returned by a subquery, whereas ALL requires the comparison to hold true against every value in the subquery output.

  SELECT product FROM inventory WHERE price > ANY (SELECT price FROM competitors);

  These operators enable flexible set-based comparison logic.
- **Common Table Expressions (WITH Clause):** A Common Table Expression (CTE) creates a temporary, named result set using the WITH clause, which can be referenced repeatedly within the main query.

  ```
  WITH max_salary AS (
     SELECT MAX(salary) AS max_sal FROM salaries
  )
  SELECT s.empno FROM salaries s
  JOIN max_salary ms ON s.salary = ms.max_sal;
  ```

  CTEs replace complex nested subqueries with clean, readable code blocks.
- **Readability and Modular Structure:** CTEs simplify query maintenance by breaking monolithic queries into sequential processing steps. They allow developers to construct multi-stage data pipelines that are easy to debug.

- **Handling Analytical Ties:** Subqueries and CTEs resolve complex analytical tasks, such as finding top-ranking records while handling ties cleanly. Computing intermediate benchmarks in a CTE allows the main query to safely filter target records.

Question 12 Record Modification & Deletion (UPDATE and DELETE FROM) and Performance

Introduction: Modifying and deleting database records are core DML operations that require optimization to minimize disk write overhead and prevent table locking.

- **Updating Records (UPDATE) Syntax:** The UPDATE statement modifies existing attribute values in a table according to specified assignment rules.

  ```
  UPDATE users SET status = 'active' WHERE last_login >= '2026-01-01';
  ```

  Omitting the WHERE clause causes the update operation to apply across every row in the target table.
- **UPDATE Performance Optimization:** UPDATE operations are relatively expensive because they require modifying table data and updating associated indexes. Performance is optimized by restricting updates exclusively to rows that actually require changes:

  ```
  -- Slower: Updates all rows indiscriminately
  UPDATE users SET email = lower(email);

  -- Faster: Updates only rows needing transformation
  UPDATE users SET email = lower(email) WHERE email != lower(email);
  ```

  Selective filtering reduces write overhead.
- **Deleting Records (DELETE FROM) Syntax:** The DELETE FROM command removes specific rows matching a predicate condition while maintaining table structures and schemas intact.

  ```
  DELETE FROM MOSQUITO_TRAP_DATA WHERE GENDER = 'Male';
  ```

  Targeted predicates prevent accidental deletion of entire datasets.
- **Full Table Clearing vs. Dropping:** Executing DELETE FROM table_name removes all data rows individually, producing an empty table structure. In contrast, DROP TABLE removes both data rows and the underlying table definition from the database catalog.
- **Indexing and Transactional Impacts:** Updating indexed columns forces the database engine to update associated index trees simultaneously, increasing processing time. When performing bulk modifications or deletions, temporary index disabling and batching commands inside explicit transactions help maintain system responsiveness.

Section C: 2-Mark Answer Bank

Question 13 Definition & History of SQL

Introduction: Structured Query Language (SQL) is the universal standardized query language for relational database systems.

- **Standard Definition:** SQL is a declarative database language designed to manage, manipulate, and query relational database management systems by issuing structured query requests.
- **Historical Origins:** Developed at IBM in the early 1970s, SQL was officially standardized by ANSI and ISO in 1986, becoming the pervasive foundational standard across modern data analytics.

Question 14 SQLite Overview and Scale Metrics

Introduction: SQLite is a lightweight, serverless database engine widely deployed across applications worldwide.

- **Architecture & Deployment:** SQLite is a self-contained, zero-configuration transactional database engine that operates in-process with a minimal memory footprint.
- **Capacity Limits:** SQLite supports impressive enterprise storage limits, including a maximum database file size of 140 terabytes and maximum individual row sizes of 1 gigabyte.

Question 15 Topic: DROP TABLE vs. DELETE FROM

Introduction: DROP TABLE and DELETE FROM perform data removal in SQL at different structural levels.

- **DROP TABLE Scope:** DROP TABLE is a DDL command that permanently purges the entire table schema definition, its data rows, and all associated indexes from the database catalog.
- **DELETE FROM Scope:** DELETE FROM is a DML command that removes specific data rows satisfying a WHERE condition while preserving the empty table structure and schema catalog.

Question 16 DISTINCT Keyword

Introduction: The DISTINCT clause eliminates duplicate rows from SQL query result sets.

- **De-duplication Functionality:** Adding DISTINCT immediately after SELECT instructs the database engine to scan retrieved column values and filter out duplicate entries.
- **Syntax Example:** SELECT DISTINCT name FROM students;

This statement returns a clean list of unique student names, filtering out repeated occurrences.

Question 17 Single Quotes vs. Backticks in SQL

Introduction: Single quotes and backticks serve distinct syntactical functions in SQL query construction.

- **Single Quotes ('...'):** Single quotes enclose literal string data values inside query clauses (e.g., WHERE last_name = 'Raymond').
- **Backticks (`...`):** Backticks enclose database, table, or column identifier names, allowing spaces or SQL reserved keywords to be used as identifiers (e.g., `MOSQUITO_TRAP_DATA`).

Question 18 Data Normalization

Introduction: Data normalization is a structural design process used to organize relational tables efficiently.

- **Definition & Purpose:** Data normalization structures database schemas to eliminate data redundancy and duplicate storage across tables.
- **Core Benefit:** In a normalized database, data changes only need to be updated in a single place rather than across multiple duplicate copies, safeguarding data consistency.

Question 19 ORDER BY Clause

Introduction: The ORDER BY clause sorts returned query records based on specified column values.

- **Sorting Direction:** ORDER BY sorts output rows in ascending order (ASC) by default unless descending order (DESC) is explicitly declared.
- **Syntax Example:** SELECT name, age FROM friends ORDER BY name ASC, age DESC;

This query orders output rows alphabetically by name and then by age in descending order.

Question 20 CROSS JOIN

Introduction: A CROSS JOIN produces the Cartesian product of two joined relational tables.

- **Cartesian Mechanics:** A CROSS JOIN pairs every individual row from the first table with every individual row from the second table without requiring join conditions.
- **Output Size Calculation:** If Table A has \(M\) rows and Table B has \(N\) rows, the resulting cross-joined dataset contains \(M \times N\) total output rows.

Question 21 SAVEPOINT and ROLLBACK TO SAVEPOINT

Introduction: Savepoints create intermediate markers within multi-statement SQL transactions.

- **Checkpoint Creation (SAVEPOINT):** Executing SAVEPOINT savepoint_name defines a specific named checkpoint within an ongoing transaction block.
- **Targeted Rollback:** Executing ROLLBACK TO SAVEPOINT savepoint_name undoes changes back to that specific marker while preserving earlier successful operations.

Question 22 INSERT INTO ... SELECT vs. SELECT INTO

Introduction: Both statements copy rows between tables but handle target table creation differently.

- **INSERT INTO ... SELECT:** Copies query results into an existing destination table that has already been created in the database schema.
- **SELECT INTO:** Dynamically creates a brand new destination table and populates it with data extracted from an existing table in a single operation.